Notas de Clase: YOLO

You Only Look Once: Detección de Objetos Unificada en Tiempo Real

Aldo Camargo, Ph.D.

July 18, 2026

Abstract

Estas notas cubren YOLO (Redmon, Divvala, Girshick y Farhadi, 2016), la arquitectura que reformuló la **detección de objetos** (localizar *y* clasificar múltiples objetos en una imagen) como un único problema de regresión resuelto por una sola red convolucional, en lugar del pipeline multi-etapa de las familias R-CNN. Derivamos la división en cuadrícula y la parametrización de cajas delimitadoras, la función de pérdida multi-término, el algoritmo de Intersección sobre Unión (IoU) y supresión de no-máximos (NMS), la arquitectura de 24 capas convolucionales inspirada en GoogLeNet, y discutimos las limitaciones de la versión original junto con la evolución hacia YOLOv2–v8 y los detectores de una sola etapa modernos.

Contents

1 De la Clasificación a la Detección de Objetos **1**

2 División en Cuadrícula y Parametrización **2**

3 Intersección sobre Unión (IoU) **3**

4 Arquitectura de la Red **3**

5 Función de Pérdida **4**

6 Limitaciones de YOLO v1 **5**

7 Resultados **5**

8 Evolución Posterior: de YOLOv2 a Detectores Modernos **6**

9 Ejercicios **6**

10 Referencias **7**

1 De la Clasificación a la Detección de Objetos

Las arquitecturas de clasificación (AlexNet, VGGNet, GoogLeNet, ResNet) producen una única etiqueta por imagen. U-Net produce una etiqueta por píxel. La **detección de objetos** exige algo intermédio: para cada objeto presente en la imagen, predecir tanto *dónde* está (una caja

delimitadora, *bounding box*) como *qué* es (una clase), y hacerlo para un número *variable* de objetos por imagen.

Definition: Detección de Objetos

Dada una imagen I, la tarea consiste en producir un conjunto de detecciones $\{(b_i, c_i, s_i)\}_{i=1}^{N}$, donde $b_i = (x, y, w, h)$ es una caja delimitadora, c_i es la clase predicha, y s_i es la confianza de la detección. El número de detecciones N varía según el contenido de la imagen.

Intuition: El enfoque anterior: R-CNN y sus variantes

Las familias R-CNN / Fast R-CNN / Faster R-CNN abordan la detección en **dos etapas**: primero, un módulo (separado o integrado) propone regiones candidatas donde podría haber un objeto; después, una segunda red clasifica y refina cada región candidata individualmente. Este enfoque es preciso pero lento — ejecutar un clasificador completo sobre miles de regiones candidatas por imagen es costoso, y las etapas no se optimizan conjuntamente de forma directa. YOLO propone lo contrario: **una sola red, una sola pasada, viendo la imagen completa exactamente una vez** (de ahí el nombre: *You Only Look Once*).

2 División en Cuadrícula y Parametrización

Definition: Cuadrícula $S \times S$

YOLO divide la imagen de entrada en una cuadrícula de $S \times S$ celdas (en la versión original, $S = 7$). **Cada celda** es responsable de predecir objetos cuyo **centro** cae dentro de esa celda — ninguna otra celda participa en esa predicción, incluso si el objeto se extiende hacia celdas vecinas.

Cada celda predice B cajas delimitadoras (en la versión original, $B = 2$), y para cada caja: coordenadas (x, y, w, h) y una **confianza**. Además, cada celda (no cada caja) predice un vector de C probabilidades de clase condicionadas a que haya un objeto en esa celda.

Definition: Puntaje de Confianza

El puntaje de confianza de una caja predicha se define como

$$\text{confianza} = \Pr(\text{Objeto}) \times \text{IoU}(\text{predicción}, \text{verdad}),$$

que debe ser cero si no hay ningún objeto en esa celda, y en caso contrario igual al IoU entre la caja predicha y la caja real.

Worked Example: Dimensión del tensor de salida

Con $S = 7$, $B = 2$, y $C = 20$ clases (como en el conjunto PASCAL VOC usado en el artículo original), el tensor de salida completo tiene dimensión

$$S \times S \times (B \times 5 + C) = 7 \times 7 \times (2 \times 5 + 20) = 7 \times 7 \times 30.$$

Cada una de las 49 celdas produce, por tanto, un vector de 30 números: 2 cajas $\times$ 5 valores $(x, y, w, h, \text{confianza})$ = 10 valores, más 20 probabilidades de clase.

Intuition: Normalización de las coordenadas de la caja

Las coordenadas (x, y) del centro de la caja se parametrizan como desplazamientos **relativos a la celda** (valores en $[0, 1]$ dentro de esa celda), mientras que (w, h) se parametrizan como fracciones **relativas al tamaño completo de la imagen**. Esta doble normalización hace que todos los valores objetivo estén acotados en $[0, 1]$, lo cual estabiliza el entrenamiento de una red que produce estas salidas mediante una capa lineal final.

3 Intersección sobre Unión (IoU)

Definition: Intersección sobre Unión

Dadas dos cajas delimitadoras A y B,

$$\text{IoU}(A, B) = \frac{\text{Área}(A \cap B)}{\text{Área}(A \cup B)} \in [0, 1].$$

Un IoU de 1 indica cajas idénticas; un IoU de 0 indica cajas sin superposición alguna.

El IoU cumple tres papeles distintos en YOLO: (1) determina la "verdad" del término de confianza durante el entrenamiento, (2) decide, quando $B > 1$ cajas compiten por el mismo objeto, cuál caja predicha es "responsable" de él (la de mayor IoU con la caja real), y (3) se usa en **supresión de no-máximos** (NMS) durante la inferencia.

Definition: Supresión de No-Máximos (NMS)

Dado un conjunto de cajas predichas con sus puntajes de confianza, NMS elimina detecciones duplicadas del mismo objeto mediante el siguiente procedimiento: (1) seleccionar la caja de mayor confianza y agregarla al resultado final; (2) eliminar todas las cajas restantes cuyo IoU con la caja seleccionada supere un umbral (típicamente 0.5); (3) repetir con las cajas restantes hasta que no quede ninguna.

4 Arquitectura de la Red

La red original consta de 24 capas convolucionales seguidas de 2 capas totalmente conectadas. El diseño está explícitamente inspirado en GoogLeNet, pero **reemplaza los módulos Inception** por una alternancia más simple de convoluciones 1×1 (reducción de canales, exactamente como en GoogLeNet) seguidas de convoluciones 3×3.

Etapa	Operaciones	Propósito
Tronco inicial	Conv $7 \times 7/2$, MaxPool 2×2	Reducción inicial de resolución
Bloques intermedios	Alternancia Conv 1×1 / Conv 3×3 (inspirado en GoogLeNet, Sección 2.2 de esas notas)	Extracción de características con costo controlado
Capas finales	2 capas totalmente conectadas	Producir el tensor $S \times S \times (B \times 5 + C)$ directamente

A diferencia de GoogLeNet (que usa *global average pooling* para eliminar las capas totalmente conectadas y así reducir parámetros), YOLO conserva capas totalmente conectadas al final porque necesita producir una salida estructurada de tamaño fijo ($7 \times 7 \times 30$) que codifica *tanto* ubicación espacial *como* contenido semántico — una tensión de diseño que las versiones posteriores de YOLO (v2 en adelante, totalmente convolucionales) resolverían de otra manera.

5 Función de Pérdida

La pérdida de YOLO es una suma ponderada de errores cuadráticos sobre varios componentes, todos calculados únicamente sobre las celdas/cajas relevantes (usando indicadoras $\mathbb{1}_{ij}^{\text{obj}}$ que valen 1 si la caja j de la celda i es responsable de un objeto).

Definition: Pérdida Multi-Término de YOLO

$$
\begin{aligned}
\mathcal{L} = {} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbb{1}_{i}^{\text{obj}} \sum_{c \in \text{clases}} (p_i(c) - \hat{p}_i(c))^2,
\end{aligned}
$$

con $\lambda_{\text{coord}} = 5$ y $\lambda_{\text{noobj}} = 0.5$ en el artículo original.

Intuition: Por qué raíz cuadrada de ancho/alto

Un error absoluto de, digamos, 10 píxeles importa mucho más para una caja pequeña (donde 10 píxeles es una fracción grande de su tamaño) que para una caja grande. Al penalizar $(\sqrt{w} - \sqrt{\hat{w}})^2$ en lugar de $(w - \hat{w})^2$ directamente, el mismo error absoluto en píxeles produce una penalización relativamente mayor cuando la caja es pequeña — la función raíz cuadrada "comprime" la escala de cajas grandes, haciendo que la pérdida sea más sensible proporcionalmente a errores en objetos pequeños.

Intuition: Por qué $\lambda_{\text{noobj}} < 1$

La gran mayoría de las $S \times S \times B$ cajas candidatas en cualquier imagen **no** contienen ningún objeto (la mayoría de las celdas de una cuadrícula 7×7 caen en el fondo). Sin ponderar hacia abajo su contribución a la pérdida, la señal de "no hay objeto aquí" dominaría numéricamente sobre la señal — mucho más escasa pero mucho más importante — de "sí hay un objeto aquí, ajusta su caja correctamente." $\lambda_{\text{noobj}} = 0.5$ reduce a la mitad esa contribución dominante.

6 Limitaciones de YOLO v1

- **Restricción espacial fuerte:** como cada celda solo predice C probabilidades de clase (no por caja, sino por celda), YOLO v1 tiene dificultad severa con **grupos de objetos pequeños y cercanos** (p. ej., una bandada de pájaros) — si sus centros caen en la misma celda, solo se puede predecir una clase para todos ellos.
- **Generalización pobre a relaciones de aspecto inusuales:** al aprender directamente las dimensiones (w, h) sin ninguna referencia estructural, la red generaliza mal a objetos con proporciones no vistas durante el entrenamiento.
- **Cuadrícula gruesa:** múltiples capas de submuestreo espacial (varias reducciones a la mitad) hacen que la característica final usada para predicción sea relativamente de baja resolución, limitando la precisión de localización de objetos pequeños.
- **Un solo IoU "responsable":** al asignar cada objeto a una sola de las B cajas por celda, las B cajas de una celda no se especializan explícitamente por tamaño o forma (a diferencia de las *cajas de anclaje*, *anchor boxes*, introducidas en YOLOv2).

7 Resultados

Modelo	FPS	mAP (VOC 2007)
Fast R-CNN	0.5	70.0
Faster R-CNN (VGG-16)	7	73.2
YOLO (base)	45	63.4
Fast YOLO (variante ligera)	155	52.7
YOLO VGG-16 (variante más precisa)	21	66.4

El resultado clave: velocidad vs. precisión

YOLO v1 no superó la precisión (mAP) de Faster R-CNN, pero la superó en velocidad por un margen enorme — suficiente para procesamiento de video en tiempo real, algo que las arquitecturas de dos etapas no podían ofrecer en 2016. Esta compensación velocidad-precisión, y cómo desplazarla favorablemente, es el hilo conductor de toda la evolución posterior de YOLO (v2 a v8+) y de la familia más amplia de "detectores de una sola etapa" (SSD, RetinaNet, etc.).

8 Evolución Posterior: de YOLOv2 a Detectores Modernos

Versión	Innovación principal
YOLOv2 (2017)	*Cajas de anclaje* (*anchor boxes*): en lugar de predecir (w, h) directamente, la red predice ajustes relativos a un conjunto de cajas de referencia con formas típicas (obtenidas por *clustering* k-means sobre las cajas del conjunto de entrenamiento). También introduce normalización por lotes en todas las capas convolucionales.
YOLOv3 (2018)	Predicción multi-escala (tres resoluciones de cuadrícula distintas, similar en espíritu a una *pirámide de características*), mejorando la detección de objetos de tamaños muy variados.
YOLOv4–v8 (2020–presente)	Backbones más eficientes (p. ej., CSPDarknet), aumento de datos más agresivo (Mosaic, MixUp), técnicas de asignación de etiquetas "sin anclas" (*anchor-free*), y optimizaciones específicas de despliegue (cuantización, exportación a formatos móviles).

A pesar de esta evolución considerable, el principio de diseño central de YOLO — **predicción directa mediante regresión sobre una cuadrícula, en una sola pasada de la red** — se mantiene sin cambios desde 2016, y sigue siendo la base conceptual de los detectores de una sola etapa más usados en aplicaciones de tiempo real hoy en día.

9 Ejercicios

1. Calcule el IoU entre dos cajas delimitadoras: $A = (x{=}50, y{=}50, w{=}100, h{=}100)$ y $B = (x{=}80, y{=}80, w{=}100, h{=}100)$ (coordenadas del centro, en píxeles). Determine si esta pareja sería suprimida por NMS con un umbral de 0.5.

2. Para $S = 7$, $B = 2$, $C = 20$, verifique que la dimensión del tensor de salida es $7 \times 7 \times 30$ (Sección 3), y explique qué representa cada uno de los 30 valores por celda.

3. Explique, usando la definición de $\mathbb{1}_{ij}^{\text{obj}}$, por qué el término de clasificación en la función de pérdida (última línea de la Sección 5) se suma **una vez por celda** y no una vez por caja, a diferencia de los términos de coordenadas y confianza.

4. Un objeto pequeño tiene $w = \hat{w} = 0.02$ (fracción del ancho de imagen) pero se predice con un error absoluto $\Delta w = 0.01$. Calcule la contribución a la pérdida usando $(w - \hat{w})^2$ directamente vs. usando $(\sqrt{w} - \sqrt{\hat{w}})^2$, y comente la diferencia relativa.

5. Investigue brevemente cómo las *cajas de anclaje* de YOLOv2 cambian la interpretación de la salida de la red respecto a YOLOv1 (Sección 8). ¿Qué limitación de la Sección 6 buscan resolver directamente?

10 Referencias

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *CVPR*, 2016.

[2] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," *CVPR*, 2017. (YOLOv2)

[3] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv:1804.02767*, 2018.

[4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *NeurIPS*, 2015.

[5] C. Szegedy et al., "Going Deeper with Convolutions," *CVPR*, 2015. (Arquitectura base que inspira el tronco convolucional de YOLO)